

Pre-training a Character-Level GPT on TinyStories under a Fixed Compute Budget

Manidip Debnath

Oakland University, Rochester, Michigan, USA
manidipdeb@oakland.edu

Abstract—We implement a nanoGPT-style decoder-only transformer from scratch (0.82 M parameters, 4 layers, width 128, one token per character) and pre-train it on 40 M characters of TinyStories. The default configuration reaches a validation loss of 0.8585 nats/char (perplexity 2.36, 1.24 bits/char), well below the 1.00 requirement. We then ask how far the loss can be pushed when every training run may use at most 2×10^{14} FLOPs. A two-stage search (12%-budget screening followed by six full-budget runs and fresh-seed replication) selects a context of 256 characters with batch size 8, which reaches **0.7016** nats/char (1.012 bits/char, perplexity 2.02) against 0.7478 for the baseline, and stays below 0.725 on all three seeds (mean 0.7060 ± 0.0038). Short-run screening ranked the winning context length last, so short-run rankings did not transfer to the full budget. Follow-up runs show that the whole gain from the longer context is explained by the way the evaluation averages over windows of the model’s own length (scored with equal windows, the longer-context model is not better than the baseline); the 0.82 M-parameter model is also close to the best size at this budget (0.48 M, 1.2 M and 1.8 M parameters are all worse). As a bonus, replacing learned position embeddings with rotary embeddings (RoPE) lowers the loss at 35% of the budget in all three seeds (mean gap 0.0079, Welch $p = 0.031$), but at the full budget RoPE is worse (0.7212 against 0.7060, $p = 0.016$).

Index Terms—language model, transformer, GPT, TinyStories, character-level tokenization, compute-constrained training, rotary position embedding.

I. INTRODUCTION

Generative pre-trained transformers (GPT) learn to predict the next token of a text and, with scale, acquire fluent language ability [1, 2]. Small models trained on simple text are a convenient laboratory: TinyStories [4] shows that children’s-story text with a limited vocabulary lets models below one million parameters write coherent stories. This work follows the nanoGPT recipe [3] and has three goals: (i) build every component of a GPT (tokenizer, causal attention, MLP, block, full model, training step) and verify it with unit tests; (ii) train a default model to a validation loss of at most 1.00 nats/char; and (iii) minimise the validation loss under a fixed budget of $C_{\max} = 2 \times 10^{14}$ training FLOPs, using a careful and honest experimental protocol. A rotary-embedding variant is studied as an optional extension.

II. DATA AND TOKENIZATION

The training text is a 40 MB subset of TinyStories (48,793 stories, 39,963,595 characters); the validation text has 27,629 stories and 22,482,104 characters. Tokenization is character-level: the vocabulary has $|V| = 96$ entries (newline plus the 95 printable ASCII characters), so the output layer is tiny (12 K parameters at $d = 128$, against 6.4 M for a 50,257-token BPE vocabulary). Fig. 1 shows the most frequent characters; the space alone accounts for about 19% of the text.

III. MODEL ARCHITECTURE

The model (Fig. 2) is a pre-norm decoder-only transformer. Token and position embeddings are summed, passed through $n_{\text{layer}} = 4$ blocks, a final LayerNorm and a linear head tied to the token embedding.

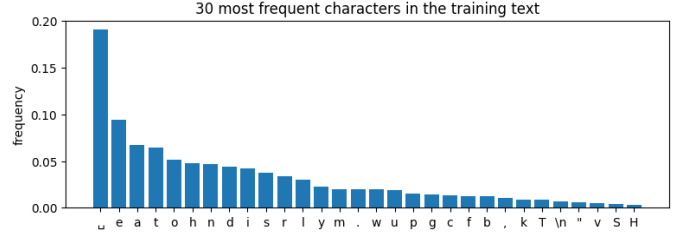


Figure 1. Thirty most frequent characters of the training text.

A) Causal self-attention

With queries, keys and values $Q, K, V \in \mathbb{R}^{T \times d_k}$ and a causal mask M ($M_{ij} = 0$ for $j \leq i$, $-\infty$ otherwise),

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V. \quad (1)$$

Multi-head attention runs $h = 4$ heads of width $d_k = d/h = 32$ in parallel with one fused projection (c_attn, width $3d$), concatenates them and applies an output matrix W_O . The mask makes future tokens invisible, which a test verifies by changing future inputs and checking that earlier logits do not change.

B) Feed-forward network and block

The MLP is $\text{MLP}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2$ with $W_1 \in \mathbb{R}^{4d \times d}$. A block applies

$$x \leftarrow x + \text{Attn}(\text{LN}_1(x)), \quad x \leftarrow x + \text{MLP}(\text{LN}_2(x)). \quad (2)$$

C) Parameter count

The estimate $12d^2$ per block gives $4 \cdot 12 \cdot 128^2 = 786,432$ plus embeddings $96 \cdot 128 + 128 \cdot 128 = 28,672$, i.e. 815,104. The model has **822,016** parameters; the difference of 6,912 is exactly the biases and LayerNorm parameters (1,664 per block $\times 4$, plus 256 for the final LayerNorm). The head is tied to the token

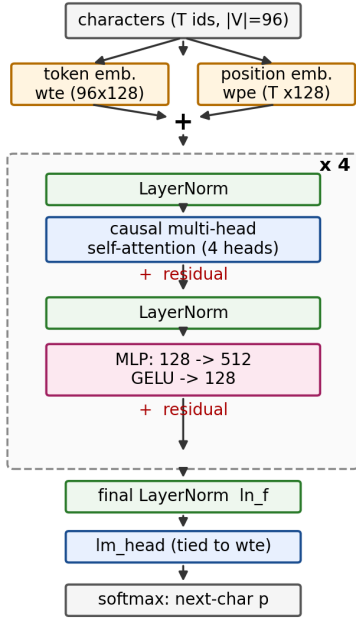


Figure 2. Architecture of the implemented GPT (default: $d = 128$, 4 heads, 4 blocks).

embedding and adds nothing. Raising the context to 256 adds 128 rows to the position table (+16,384 parameters), giving 838,400.

IV. TRAINING PROCEDURE

A) Optimisation

We use AdamW ($\beta = (0.9, 0.99)$) [7] with weight decay 0.1 on matrices and embeddings only, gradient clipping at norm 1.0, and a linear warm-up (100 steps) followed by cosine decay from $\eta_{\max} = 3 \times 10^{-3}$ to $\eta_{\min} = 3 \times 10^{-4}$. A training step is: forward pass, clear gradients, backward pass, optimizer update.

B) Compute accounting

Training compute is estimated as [5]

$$C \approx (6N + 12n_{\text{layer}}dT) \times D, \quad D = \text{steps} \times B \times T, \quad (3)$$

where N is the parameter count, B the batch size and T the context length. The attention term grows with T , so at a fixed C a longer context sees fewer tokens. A budget planner converts a fraction of $C_{\max}$ into a number of steps with exactly this formula, so no run can exceed the budget.

C) Default run

The default model (3,000 steps, batch 32, $T = 128$) uses 7.0×10^{13} FLOPs (35% of the budget) and trains in 0.8 minutes on a Tesla T4. Training and validation loss fall together from $\ln 96 \approx 4.56$ (Fig. 3); an early smoke test checks that 200 steps already bring the loss below 2.5. Fig. 3 (left) illustrates the schedule shape.

V. EVALUATION PROTOCOL

A) Fixed-slice loss

The training-time `estimate_loss` averages random batches and is noisy (standard error about 0.003–0.004). All compar-

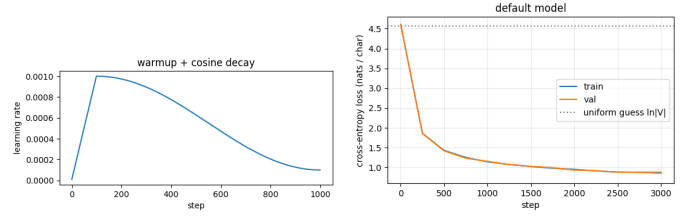


Figure 3. Left: shape of the warm-up and cosine-decay schedule (unit-test plot with $\eta_{\max} = 10^{-3}$, 1,000 steps). Right: training and validation loss of the default model; the dotted line is the uniform guess $\ln|V|$.

Table I
BASELINES AND MODELS ON THE FIXED VALIDATION SLICE

Model	Loss	Perplexity	bits/char	Compr.
Uniform	4.5643	96.00	6.585	1.2×
Unigram	3.0806	21.77	4.444	1.8×
Bigram	2.2800	9.78	3.289	2.4×
Default GPT (3,000 steps)	0.8585	2.36	1.239	6.5×
Final GPT (full budget)	0.7016	2.02	1.012	7.9×

isons therefore use a deterministic *fixed-slice* loss: the first 10^6 validation characters are cut into consecutive windows of the model’s own `block_size`, and the mean cross-entropy over all windows is reported. Part 8 of the assignment uses the same slice.

B) Reference baselines

Table I gives context-free references computed from training counts, and the default and final models, in three units: nats/char, perplexity $e^{\mathcal{L}}$ and bits/char $\mathcal{L}/\ln 2$ (compression is relative to 8-bit ASCII).

VI. EXPERIMENTS UNDER THE COMPUTE BUDGET

A) Search protocol

Fig. 4 summarises the protocol. The rule for choosing the final model was fixed before the runs: take the setting with the lowest fixed-slice loss at the full budget (seed 1337), then replicate the winner and the baseline with two fresh seeds, because selecting on one seed is biased in favour of the winner. All Part 7 models are the assignment’s GPT trained with `train()`; only the knobs context length, batch size and learning rate are changed, and every run uses at most $0.995C_{\max}$.

B) Screening at 12% of the budget

Four candidates were trained for 12% of the budget (Table II, Fig. 5). A candidate had to beat the baseline by a margin of 0.010 to be chosen in advance; none did (best: weight decay 0.01, -0.0070), so screening alone would have kept the baseline. The context-256 candidate ranked *last* (+0.1772).

C) Full-budget runs

Because short runs can mislead, six settings were then trained with 99.5% of the budget (about 2.3–2.9 minutes each on a T4). Table III gives the result and Fig. 6 the learning curves.

Three observations follow. (1) Moving from context 128 to 256 at equal tokens per step (batch 16) lowers the loss by 0.0135. (2) Halving the batch to 8 doubles the number of updates and lowers it by a further 0.0327; a higher learning rate at batch 16 gives most of the same gain (0.7067). ‘T256 bs8’ sees fewer

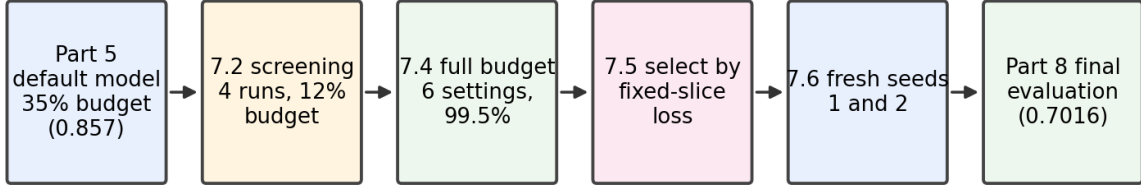


Figure 4. Experimental workflow of Part 7.

Table II
SCREENING RUNS (12% BUDGET, FIXED-SLICE LOSS)

Candidate	Loss	vs. base
wd 0.01	1.0979	-0.0070
base ($\eta = 3e-3$)	1.1050	0
$\eta = 2e-3$	1.1527	+0.0478
context 256 (batch 16)	1.2821	+0.1772

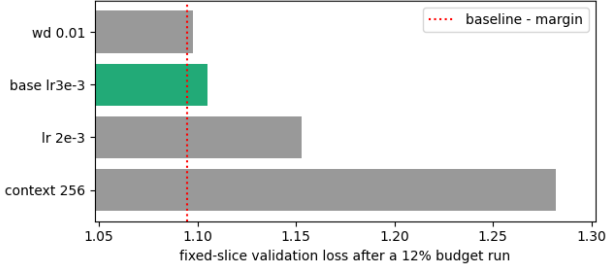


Figure 5. Screening result. The red line is the baseline minus the 0.010 margin.

characters than the baseline (30.1 M against 34.8 M) but performs 1.7 times as many updates. This is not a general rule: at context 128 a batch of 8 is *worse* than 32 (Section F). (3) Contexts longer than 256 did not pay off: the attention term makes tokens more expensive, so only 8,650 (T384) and 5,802 (T512) steps fit. Overall the run table and the compute plot are shown in Fig. 7.

D) Fresh seeds

Table IV replicates the baseline and the winner with seeds 1 and 2. On the two fresh seeds the baseline averages 0.7544 and the winner 0.7082, a gain of 0.0461, about 12 times the larger seed standard deviation (0.0039). All three seeds of the winner are below the 0.725 threshold (worst 0.7088, margin 0.016), so the tier does not depend on the seed. The winner was chosen among six settings on the same slice it is scored on, which favours it slightly; this is why the fresh seeds are reported.

E) Loss by position and the scoring artefact

Fig. 8 plots the loss against the position inside the window. At position 0 the model has seen one character and its loss (about 2.3) is close to the bigram baseline (2.28); after about ten characters it is near 1.0, and it keeps falling slowly. The final model (context 256) lies below the default one at almost every position. Because the evaluation averages over whole windows of the model’s own length, a longer window contains a

Table III
FULL-BUDGET RUNS (SEED 1337, FIXED-SLICE LOSS)

Setting	Params	Steps	Loss	Δ
T256, batch 8	838,400	14,715	0.7016	-0.0462
T256, batch 16, $\eta=4e-3$	838,400	7,357	0.7067	-0.0411
T384, batch 8	854,784	8,650	0.7211	-0.0267
T512, batch 8	871,168	5,802	0.7251	-0.0227
T256, batch 16	838,400	7,357	0.7343	-0.0135
T128, batch 32 (baseline)	822,016	8,495	0.7478	0

Table IV
SEED REPLICATION (FIXED-SLICE LOSS)

Setting	1337	1	2	Mean	Std
T128 bs32	0.7478	0.7535	0.7553	0.7522	0.0039
T256 bs8	0.7016	0.7077	0.7088	0.7060	0.0038

smaller share of the expensive first characters, which lowers the *measured* loss by itself. Section F quantifies this.

F) Follow-up runs: scoring, model size, dropout

Four questions were left open by the search and are answered with additional full-budget runs (seed 1337, 99.5% of $C_{\max}$, `train()`, learning rate 3×10^{-3} without re-tuning), in the same session as the rest of the notebook. The baseline and the winner are reused from the search. The baseline (seed 1337) is not exactly reproducible: it scored 0.7478 here and 0.7513 and 0.7552 in two earlier sessions, so one baseline value carries an uncertainty of about ± 0.004 ; the winner reproduced 0.7016 exactly every time.

Scoring effect. The same trained models were scored with windows of different lengths (Table V). With equal 128-character windows the winner (0.7532) is *not* better than the baseline (0.7478). The Part 8 difference of 0.0462 splits exactly into -0.0516 from the window length alone (112% of the gain) and $+0.0054$ from the model, which is within the run-to-run variation of the baseline: the window length explains the whole gain. On windows of 32 and 64 characters the winner is worse, as expected for a model trained only on 256-character windows. The longer context does not make the model a better predictor at a given context length; it lowers the metric, which averages over whole windows. This is allowed by the specified evaluation, and an evaluation with overlapping windows would remove it (not run).

Model size. Table VI and Fig. 9 change the size around the winner. The loss is U-shaped: a smaller model that sees more

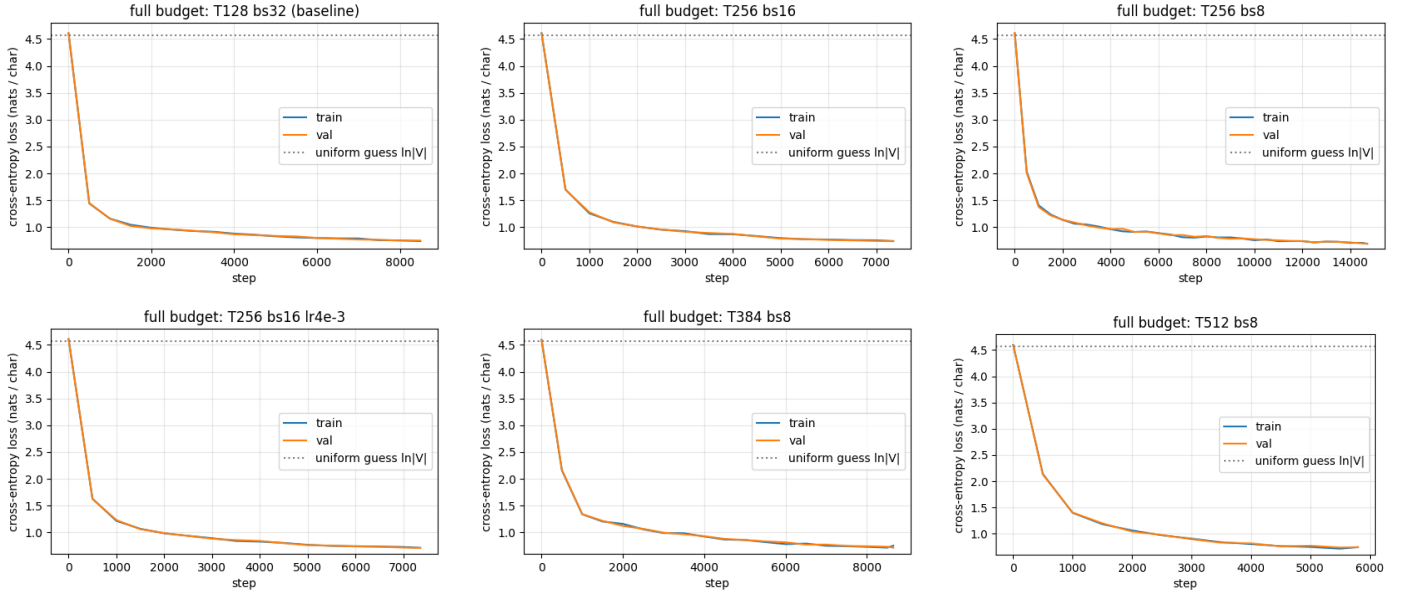


Figure 6. Training and validation loss of the six full-budget runs (top: T128 bs32 baseline, T256 bs16, T256 bs8; bottom: T256 bs16 $\eta=4e-3$, T384 bs8, T512 bs8).

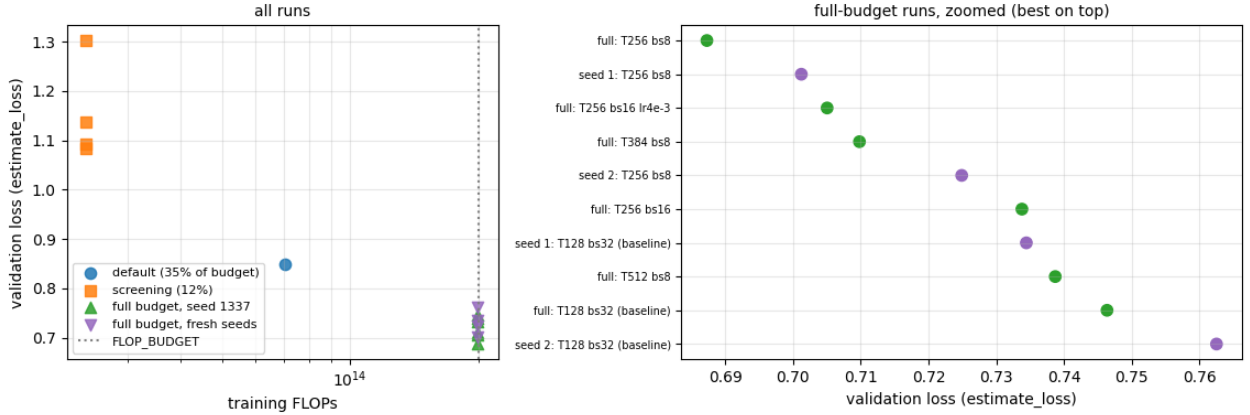


Figure 7. Validation loss (from `estimate_loss`) against training FLOPs for every run. Left: all runs; right: the nine full-budget runs ranked, best on top (green: seed 1337, purple: fresh seeds).

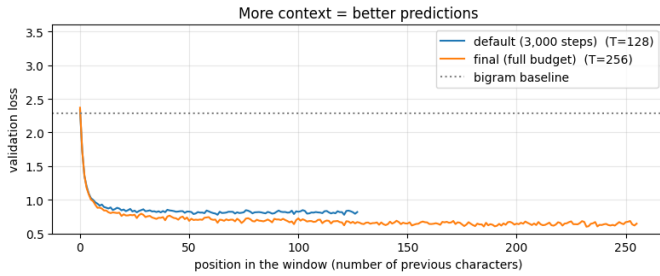


Figure 8. Fixed-slice loss by position in the window for the default (T=128) and final (T=256) models.

text (width 96, 102 characters per parameter) is worse by 0.0275, a deeper one (6 layers, 16.5) by 0.0113, and a wider one (width 192, 8) by 0.1126. At about 36 characters per parameter the winner is close to the best trade-off among the sizes tried, so neither a bigger model nor more tokens paid off. The learning rate was not re-tuned per size, so the width-192 result may partly

Table V
LOSS OF THE SAME MODELS SCORED WITH DIFFERENT WINDOW LENGTHS

Model (block size)	32	64	128	256
T128 bs32 baseline (128)	0.9033	0.8077	0.7478	–
T256 bs8 winner (256)	0.9274	0.8246	0.7532	0.7016

be a learning-rate effect. *Context against batch.* Context 128 with batch 8 (33,983 steps) gives 0.7700, worse than batch 32 (0.7478), so the smaller batch does not help by itself. *Dropout.* Dropout 0.1 at the winning setting gives 0.7826, worse by 0.0810, consistent with the absence of overfitting.

G) Samples and temperature

Generation divides the logits by a temperature τ : $p_i = e^{z_i/\tau} / \sum_j e^{z_j/\tau}$. At $\tau = 0.5$ the distribution is sharpened: every word is real and the sentences are short and mostly grammatical, but words and phrases repeat (“very happy”, “so happy”, “a big street”, “a big friend”). At $\tau = 1.0$ names and sentence shapes

Table VI
SAME BUDGET, DIFFERENT SIZES (CONTEXT 256, BATCH 8)

Setting	Params	Tokens (M)	Tok./param	Loss
width 96	481,344	48.9	102	0.7292
width 128 (winner)	838,400	30.1	36	0.7016
6 layers	1,234,944	20.4	16.5	0.7129
width 192	1,847,424	14.8	8	0.8142
dropout 0.1	838,400	30.1	36	0.7826
T128, batch 8	822,016	34.8	42	0.7700

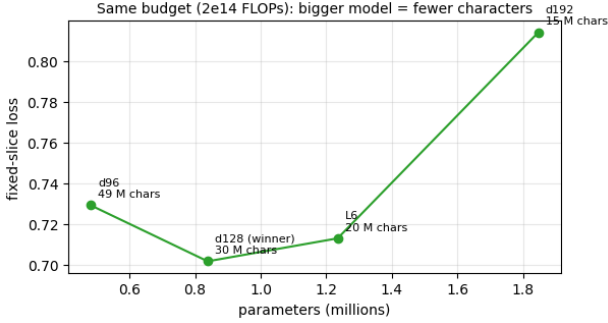


Figure 9. Loss against model size at the same compute; the labels give the number of training characters.

are plausible but non-words and grammar errors appear (“airor”, “Miamia”, “He also laughed and happy to read”). At $\tau = 1.5$ the distribution is flat and errors compound, producing non-words (“broes”, “olderful”, “Ectefully”) and misplaced punctuation. A vocabulary check shows that 99.1% of the 845 words generated by the final model at $\tau = 0.8$ occur at least three times in the training text, against 98.7% of 855 for the default model; this difference is not significant (one standard error of a rate near 99% is 0.4–0.5 points), and the metric cannot measure coherence. As expected for a base model, it *continues* text instead of answering: after “The capital of France is” it writes a made-up word and a story (“cluddy. Mia was sad...”), because its only training objective is next-character prediction on stories.

VII. BONUS: ROTARY POSITION EMBEDDINGS

Rotary embeddings (RoPE) [6] remove the learned position table. Each head’s query and key are rotated by position-dependent angles before the scores are computed, so $\langle R_{iq}, R_{jk} \rangle$ depends only on $i - j$; the value vector is left unchanged. The RoPE model has 805,632 parameters (16,384 fewer, exactly the removed table). Four tests pass: position 0 is not rotated, rotations preserve vector length, scores are unchanged when both positions shift by the same amount, and attention remains causal.

At the default setting (35% of the budget, same configuration, learning rate, seed and data), RoPE reaches a fixed-slice loss of 0.8442 against 0.8585 for learned positions (paired difference 0.0143 ± 0.0005 over 7,812 windows, RoPE better on 61.9%). Its validation loss is lower at every logged step, most strongly early (1.41 against 1.85 at step 250; Fig. 10).

A seed replication (Table VII, Fig. 11) gives 0.8476 ± 0.0032 for RoPE against 0.8555 ± 0.0026 for learned positions; the mean gap of 0.0079 is 3.3 standard errors of the seed spread, and every RoPE seed is below every learned-position seed (proba-

Table VII
RoPE AGAINST LEARNED POSITIONS (35% BUDGET)

Model	1337	1	2	Mean	Std
Learned	0.8585	0.8544	0.8536	0.8555	0.0026
RoPE	0.8442	0.8480	0.8506	0.8476	0.0032

bility 1/20 if the runs were exchangeable). A Welch t -test gives $p = 0.031$, so the evidence is moderate, not conclusive. The advantage does *not* carry over to the full budget. With the Part 7 winner setting (context 256, batch 8, the same 14,715 steps, three seeds), RoPE scores 0.7212 ± 0.0050 against 0.7060 ± 0.0038 for learned positions (Welch $p = 0.016$; every learned-position seed, 0.7016–0.7088, is below every RoPE seed, 0.7160–0.7260). The 35% result is therefore best read as “RoPE learns faster early”. Limits: the learning rate was not re-tuned for RoPE and we did not investigate the reason; there are three seeds per model; and the RoPE model is not the Part 7 model, since the final model must come from `train()`, which builds the original GPT.

VIII. DISCUSSION AND LIMITATIONS

Compute mattered most: using the full budget instead of 35% lowers the baseline from 0.8585 to 0.7478. At a fixed budget the best setting under the Part 8 metric was a longer context with a smaller batch, but the whole advantage is explained by the window-length effect, not by a better model, and a smaller batch is not generally better (it is worse at context 128). Model size around the winner is close to optimal and dropout hurts. Short-run (12%) rankings failed to predict this, so comparisons were made at the full budget. No overfitting occurs: the final run processes 30.1 M characters, 0.75 of one pass over the 40 M training characters, and training and validation loss are equal at the last step (0.688 and 0.687). Limitations: one hardware type (T4, GPU runs are not bit-for-bit reproducible, so a re-run changes the last digits); selection on a single slice; the window-length effect described above; run-to-run variation of the baseline of about ± 0.004 ; one seed for each follow-up run and no learning-rate re-tuning for other sizes or for RoPE; no evaluation with overlapping windows.

IX. CONCLUSION

A from-scratch character-level GPT with 0.82 M parameters reaches 0.8585 nats/char with the default setting and **0.7016** nats/char (perplexity 2.02, 1.01 bits/char, $7.9\times$ compression) under a 2×10^{14} -FLOP budget, with context 256 and batch 8. The gain over the baseline is 12 times larger than the seed noise, but all of it is the effect of scoring longer windows, not a better model. Rotary embeddings help at 35% of the budget (0.8476 against 0.8555) but not at the full budget (0.7212 against 0.7060).

ACKNOWLEDGMENT

This report accompanies the notebook `nanogpt_pretraining_final_Manidip_Debnath_v5.ipynb`; all numbers are taken from its executed outputs (Google Colab Tesla T4; Appendix B was run in a second session).

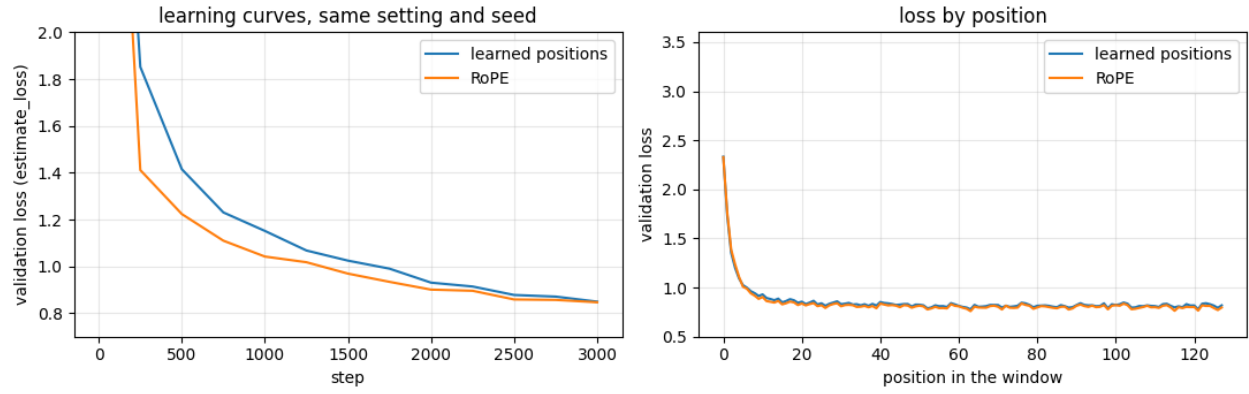


Figure 10. RoPE against learned positions at the default setting. Left: learning curves; right: loss by position.

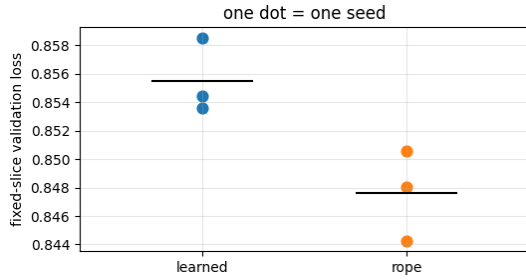


Figure 11. One dot per seed; black bars are means.

REFERENCES

- [1] A. Vaswani *et al.*, “Attention is all you need,” in *Proc. NeurIPS*, 2017.
- [2] A. Radford *et al.*, “Language models are unsupervised multitask learners,” OpenAI, 2019.
- [3] A. Karpathy, “nanoGPT,” 2022. [Online]. Available: <https://github.com/karpathy/nanoGPT>
- [4] R. Eldan and Y. Li, “TinyStories: How small can language models be and still speak coherent English?” arXiv:2305.07759, 2023.
- [5] J. Kaplan *et al.*, “Scaling laws for neural language models,” arXiv:2001.08361, 2020.
- [6] J. Su *et al.*, “RoFormer: Enhanced transformer with rotary position embedding,” arXiv:2104.09864, 2021.
- [7] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.